Partial Order Reduction (POR)

The Complete Guide to Strong Stubborn Sets

Lecture 6: Machine Intelligence

January 4, 2026

Contents

1 The Big Picture: Why Do We Need Partial Order Reduction? 2
- 1.1 The Course Red Thread So Far . . . 2
- 1.2 The Gripper Problem: A Shocking Discovery . . . 4
- 1.3 Almost Perfect Heuristics: The Formal Definition . . . 5
- 1.4 The Search Space Explosion: Why It Happens . . . 6

2 Pruning Methods: State vs. Action Pruning 7
- 2.1 The Two Flavors of Pruning . . . 7
- 2.2 The Socks and Shoes Example: POR in Action . . . 8

3 Action-Pruning Functions: The Formal Framework 12
- 3.1 Basic Definitions . . . 12
- 3.2 Pruned State Space . . . 13
- 3.3 Admissible Action-Pruning Functions . . . 14
- 3.4 The Key Theorem for Admissibility . . . 15

4 Strong Stubborn Sets: Ingredients (The Building Blocks) 17
- 4.1 The Running Example: 1/2-Log . . . 17
- 4.2 Action Dependencies: Enabling . . . 18
- 4.3 Action Dependencies: Disabling . . . 19
- 4.4 Action Dependencies: Conflict . . . 20
- 4.5 Action Dependencies: Interfering . . . 21
- 4.6 Action Dependencies: Commutative . . . 22
- 4.7 Summary of Action Dependencies . . . 24
- 4.8 Necessary Enabling Sets . . . 25

5 Strong Stubborn Sets: Theory 28
- 5.1 The Intuition Behind Strong Stubborn Sets . . . 28
- 5.2 The Formal Definition of Strong Stubborn Sets . . . 30
- 5.3 S3 Pruning Function Definition . . . 32
- 5.4 The Main Theorem: S3 is Admissible! . . . 33
- 5.5 Proof of S3 Admissibility - Part 1 . . . 34
- 5.6 Proof of S3 Admissibility - Part 2: Claim (a) . . . 35
- 5.7 Proof of S3 Admissibility - Part 3: Claim (b) . . . 36
- 5.8 Proof of S3 Admissibility - Part 4: Claim (c) . . . 37
- 5.9 Proof of S3 Admissibility - Part 5: The Final Step . . . 38

6 Strong Stubborn Sets: Practice (Making It Work) 39
6.1 The Algorithm: From Definition to Implementation . . . 39
6.2 The Two Big Challenges for Implementation . . . 40
6.3 Syntactic Characterization of Disabling . . . 41
6.4 Syntactic Characterization of Conflict . . . 42
6.5 Finding Interfering Actions: The Complete Picture . . . 43
6.6 The Hard Part: Necessary Enabling Sets . . . 44
6.7 Approximating Necessary Enabling Sets . . . 45
6.8 Choosing Which Fact to Focus On . . . 47
6.9 Complete S3 Example: Three Little Pigs . . . 48
6.10 How Much Does the Choice Matter? . . . 51

7 Summary and Key Takeaways 52
7.1 The Big Picture: What We Learned . . . 52
7.2 Key Concepts Recap . . . 54
7.3 Practical Guidelines . . . 58
7.4 Common Pitfalls and How to Avoid Them . . . 59
7.5 Quick Reference: The S3 Algorithm . . . 60
7.6 Further Reading and Advanced Topics . . . 61

1 The Big Picture: Why Do We Need Partial Order Reduction?

1.1 The Course Red Thread So Far

The Story Up Until Now

Where We've Been:

1. **Lectures 1-2:** Uninformed search - BFS, DFS, Dijkstra, A*
   - These systematically explored state spaces without domain knowledge
   - Optimal algorithms (like Dijkstra/UCS) guaranteed shortest paths
2. **Lecture 3:** Heuristic search - Using $h(n)$ to guide search toward goals
   - A* combines $g(n)$ (cost so far) with $h(n)$ (estimate to goal)
   - Admissible heuristics guarantee optimal solutions
3. **Lecture 4:** Classical planning & FDR representation
   - Structured representations with variables, preconditions, effects
   - Delete relaxation gives admissible heuristics (like h^+)
4. **Lecture 5:** Pattern Database Heuristics - Abstraction-based heuristics
   - Create simplified versions of the problem
   - Precompute exact solutions to abstractions
   - Very powerful heuristics, can be almost perfect!

The Shocking Discovery We Face:
Even with **ALMOST PERFECT** heuristics (off by only 1!), we can *still* have exponential search spaces!

Why? The Culprit: Plan Permutations
Imagine you need to put on socks and shoes for both feet:

- ⟨sock_L, shoe_L, sock_R, shoe_R⟩ - One valid plan
- ⟨sock_R, sock_L, shoe_L, shoe_R⟩ - Another valid plan
- ⟨sock_L, sock_R, shoe_R, shoe_L⟩ - Yet another valid plan
- ... and 3 more permutations!

Key insight: All 6 plans achieve the same goal with the same cost! They're just different orderings of the same actions.
When actions are **independent** (like putting on left vs. right sock), any ordering works. But A* with an almost-perfect heuristic will still explore states from *all these permutations* because they all have $g(n) + h(n)$ values close to optimal!

This Lecture's Solution: A New Dimension
Partial Order Reduction (POR) - Works *orthogonally* to heuristics!

- Heuristics (Lectures 3-5): Guide search *toward* the goal
- POR (Lecture 6): Prune different *orderings* of the same actions

Think of it as: Heuristics tell us "which direction to go," while POR tells us "which equivalent paths to ignore."

1.2 The Gripper Problem: A Shocking Discovery

Even Almost-Perfect Heuristics Can Fail!

The Gripper Domain:
You have a robot with two grippers. The robot needs to transport n balls from location L to location R.

The Shocking Result:

Number of balls	Optimal cost	Nodes with $h^*(n) - 1$
4	11	125
6	17	925
8	23	5,885
10	29	34,301
12	35	188,413
14	41	991,229
16	47	5,046,269

What does this mean?
Even if our heuristic is off by ONLY 1 (meaning $h(n) = h^*(n) - 1$), we still need to explore MILLIONS of nodes! This grows **exponentially** with the problem size!

Why does this happen? Let's think through the Gripper domain:

- Robot can carry 2 balls at a time (one in each gripper)
- To transport 4 balls optimally: pick up 2, move, drop 2, return, pick up 2, move, drop 2
- Optimal cost: 11 actions (load, load, move, unload, unload, move, load, load, move, unload, unload)

But consider a state halfway through an optimal plan where 2 balls are at L and 2 balls are at R:

- We could have picked up ball_1 then ball_2, or ball_2 then ball_1
- We could have dropped ball_3 then ball_4, or ball_4 then ball_3
- Each choice leads to a *different state*, but all are equally good!

The problem: Even though these orderings don't matter (the balls are interchangeable), A* doesn't know that! It treats each different ordering as a separate state that needs to be explored.

Consequence: With n balls, the number of "essentially equivalent" states grows exponentially, and even an almost-perfect heuristic won't save us because all these states have similar $g(n) + h(n)$ values!

1.3 Almost Perfect Heuristics: The Formal Definition

Almost Perfect Heuristic

Plain English Definition:
A heuristic h is **almost perfect** if it's never more than some small constant c away from the true optimal cost h^*.

Mathematical Definition:
Let Π be a planning task with states S, and let h be an admissible heuristic for Π.
We say that h is **almost perfect** if there exists $c \in \mathbb{N}_0^+$ (a non-negative integer) such that:

$$h^*(s) - h(s) \leq c \quad \text{for all states } s \in S$$

Breaking Down the Notation:

- $h^*(s)$ = The TRUE optimal cost from state s to the goal
- $h(s)$ = Our heuristic's estimate of the cost from s to the goal
- $h^*(s) - h(s)$ = The "error" in our heuristic
- c = The maximum error (a constant bound)

Examples:

- If $c = 0$: Perfect heuristic ($h = h^*$) - knows exact costs!
- If $c = 1$: At most 1 off from perfect - incredibly accurate!
- If $c = 2$: At most 2 off from perfect - still amazing!

Connection to A*: Recall from Lecture 3 that A* expands nodes n where $g(n)+h(n) \leq h^*(I)$, where I is the initial state. If $h(n) = h^*(n) - c$, then:

$$g(n) + (h^*(n) - c) \leq h^*(I)$$

This means A* must expand *all* nodes n where $g(n) + h^*(n) \leq h^*(I) + c$.
When $c = 1$, this includes all nodes on optimal paths *plus* nodes one step off optimal paths!

The Shocking Result: Even with $c = 1$ (essentially perfect!), we can still have exponential explosion due to plan permutations!

1.4 The Search Space Explosion: Why It Happens

The Root Cause: Plan Permutations

The Core Problem:
When actions are **independent** (don't affect each other), they can be executed in any order, leading to the same result.

Gripper Example Visualized:
State: Half the balls at L, half at R (all on optimal path!)
From this state, we can:

- Pick up any ball at L
- Drop any ball at R
- Move the robot

Every combination leads to different intermediate states, but they're all equally good!

The Mathematical Reality:
Define $N^c(\Pi)$ = number of nodes where $g(n) + (h^*(n) - c) < h^*(I)$

This is the number of nodes that A* MUST expand when using an almost-perfect heuristic with error c.

Results:

- If state space is a tree: $N^c(\Pi)$ is linear [Gaschnig 1977]
- In standard planning domains: $N^c(\Pi)$ grows **exponentially** even for $c = 1$! [Helmert & Röger 2008]

The Bottom Line: Heuristics alone cannot save us from exponential explosion caused by plan permutations. We need a different approach!

2 Pruning Methods: State vs. Action Pruning

2.1 The Two Flavors of Pruning

Optimality-Preserving Pruning Methods

The Goal: Reduce search effort WITHOUT losing the ability to find optimal solutions. This is different from heuristics! Heuristics guide which direction to explore; pruning decides which expansions to skip entirely.

Two Main Approaches:

1. **State Pruning:**

   What it does: Compares states and prunes entire states whose exploration can be shown to be unnecessary.

   How it works: Looks across different states and says "State B is dominated by State A, so we don't need to explore B."

   Example: Duplicate detection, symmetry reduction

   When it helps: When many states are "essentially the same"

2. **Action Pruning:**

   What it does: For each state, analyzes which applicable actions actually need to be explored.

   How it works: In state s with 10 applicable actions, identify that we only need to explore 3 of them to maintain optimality.

   Example: **Partial Order Reduction** (this lecture!)

   When it helps: When many actions are "interchangeable" or lead to equivalent situations

This Lecture Focuses On:
Partial Order Reduction (POR) - An action pruning method that identifies and prunes permutable parts of the search space.

Key Insight: Instead of exploring all possible orderings of independent actions (like left-sock, right-sock vs. right-sock, left-sock), POR lets us pick one arbitrary ordering and safely ignore the others!

Why "Partial Order"? Because we're recognizing that some actions don't have a necessary ordering between them - they can be done in any order. We're moving from a "total order" (one specific sequence) to a "partial order" (some ordering constraints, but not all).

2.2 The Socks and Shoes Example: POR in Action

The Intuition Behind POR

Problem: Put on socks and shoes for both feet.
Actions:

- sock_L (put on left sock)
- sock_R (put on right sock)
- shoe_L (put on left shoe)
- shoe_R (put on right shoe)

All Possible Valid Plans:

1. ⟨sock_L, shoe_L, sock_R, shoe_R⟩
2. ⟨sock_L, sock_R, shoe_L, shoe_R⟩
3. ⟨sock_L, sock_R, shoe_R, shoe_L⟩
4. ⟨sock_R, shoe_R, sock_L, shoe_L⟩
5. ⟨sock_R, sock_L, shoe_L, shoe_R⟩
6. ⟨sock_R, sock_L, shoe_R, shoe_L⟩

All these plans have the same optimal cost: 4 actions.

Step 1: Understanding the Full Search Space WITHOUT POR

Let's visualize what happens when we explore ALL possible action sequences:

Initial State: Nothing on feet
Applicable Actions: sock_L, sock_R (shoes require socks first!)

Level 0 (Initial):

```
State I: {nothing on}
```

Level 1: From I, we can do sock_L OR sock_R

```
State A: {sock_L on}     (via sock_L)
State B: {sock_R on}     (via sock_R)
```

Level 2: From each state, multiple options:

```
From State A:
  → State C: {sock_L on, shoe_L on}     (via shoe_L)
  → State D: {sock_L on, sock_R on}     (via sock_R)

From State B:
  → State E: {sock_R on, shoe_R on}     (via shoe_R)
  → State F: {sock_R on, sock_L on}     (via sock_L)
```

KEY OBSERVATION: States D and F are **identical**! Both have both socks on. This is because sock_L and sock_R are **commutative** - they don't interfere with each other.

Level 3: Continuing from all states:

```
From State C: {sock_L, shoe_L on}
  → State G: {sock_L, shoe_L, sock_R on}   (via sock_R)

From State D/F: {sock_L, sock_R on}
  → State H: {sock_L, sock_R, shoe_L on}   (via shoe_L)
  → State J: {sock_L, sock_R, shoe_R on}   (via shoe_R)

From State E: {sock_R, shoe_R on}
  → State K: {sock_R, shoe_R, sock_L on}   (via sock_L)
```

Level 4: Finally reaching the goal:

```
All paths converge to: {sock_L, sock_R, shoe_L, shoe_R on}
```

Total Nodes Explored Without POR:

- Level 0: 1 state
- Level 1: 2 states
- Level 2: 4 states (but 2 duplicates!)
- Level 3: 4 states
- Level 4: 1 state (goal)

Even with duplicate detection, we explore many redundant paths!

Step 2: Identifying Action Dependencies

POR analyzes which actions interfere with each other:

Action Pair	Commutative?	Why?
sock_L, sock_R	YES	Don't affect each other's preconditions/effects
shoe_L, shoe_R	YES	Don't affect each other (assuming socks on)
sock_L, shoe_L	NO	sock_L enables shoe_L (dependency!)
sock_R, shoe_R	NO	sock_R enables shoe_R (dependency!)
sock_L, shoe_R	YES	Independent feet
sock_R, shoe_L	YES	Independent feet
shoe_L, sock_R	YES	Independent feet
shoe_R, sock_L	YES	Independent feet

POR's Insight: When actions are commutative, we only need to explore ONE ordering, not all permutations!

Step 3: How POR Prunes the Search Space

POR works by selecting a **stubborn set** of actions at each state - a subset that preserves optimality.

At Initial State I:

- **Without POR:** Explore both sock_L and sock_R
- **With POR:** Pick ONE commutative action, say sock_L
- **Reasoning:** Since sock_L and sock_R are commutative and both lead to equivalent states, we can arbitrarily choose one. We'll eventually get sock_R done after sock_L.

At State A: {sock_L on}

- **Applicable actions:** shoe_L, sock_R
- **POR Analysis:** These actions are independent (commutative)
- **POR Decision:** Pick ONE, say sock_R
- **Reasoning:** We need both eventually; order doesn't matter

At State D: {sock_L, sock_R on}

- **Applicable actions:** shoe_L, shoe_R
- **POR Analysis:** These actions are commutative
- **POR Decision:** Pick ONE, say shoe_L

At State H: {sock_L, sock_R, shoe_L on}

- **Applicable actions:** shoe_R (only one left!)
- **POR Decision:** Must take it

Step 4: The Pruned Search Space

With POR, the search follows a SINGLE path:

Pruned Search Path:
State I: {}
↓ (POR chooses: sock_L only)
State A: {sock_L}
↓ (POR chooses: sock_R only)
State D: {sock_L, sock_R}
↓ (POR chooses: shoe_L only)
State H: {sock_L, sock_R, shoe_L}
↓ (Only option: shoe_R)
State GOAL: {sock_L, sock_R, shoe_L, shoe_R}

Nodes Explored:

- Without POR: ∼12 nodes (with duplicate detection)

- With POR: 5 nodes (one path!)

The Magic: POR guarantees that ANY other valid ordering we could have chosen would lead to an equivalent or worse solution. By pruning commutative action orderings, we explore only ONE representative path instead of ALL permutations!

Why This Works

Optimality is Preserved Because:

1. All 6 valid plans have the same cost (4 actions)
2. POR only prunes paths that differ by the ORDER of commutative actions
3. Commutative actions don't affect each other's costs or applicability
4. The pruned path finds one of the optimal solutions
5. Any other optimal solution would be reached by just swapping commutative actions

The Key Property: If two actions are commutative and both applicable, we only need to explore ONE ordering to find an optimal solution!

3 Action-Pruning Functions: The Formal Framework

3.1 Basic Definitions

Action-Pruning Function

Plain English:
An action-pruning function is a rule that tells us, for each state, which subset of applicable actions we should actually consider.
Think of it as a "filter" that takes all possible actions in a state and returns only the ones we need to explore.

Formal Definition:
Let Π be a planning task with states S and actions A.
An **action-pruning function** for Π is a function:

$$\rho : S \mapsto \mathcal{P}(A)$$

such that for all states $s \in S$:

$$\rho(s) \subseteq \mathcal{A}(s)$$

Breaking Down the Notation:

- ρ (rho) = The action-pruning function (our "filter rule")
- S = Set of all states
- $\mathcal{P}(A)$ = Power set of actions (all possible subsets of actions)
- $\mathcal{A}(s)$ = Set of actions applicable in state s (recall from Lecture 4: actions whose preconditions are satisfied)
- $\rho(s)$ = The subset of applicable actions we'll actually explore in state s

Example:
State s has 10 applicable actions: $\mathcal{A}(s) = \{a_1, a_2, \ldots, a_{10}\}$
Our pruning function says: $\rho(s) = \{a_1, a_3, a_7\}$
This means we only explore 3 out of the 10 applicable actions!

Connection to Search: When using A* or any search algorithm:

- *Without pruning:* When expanding state s, generate successors for all actions in $\mathcal{A}(s)$
- *With pruning:* When expanding state s, generate successors only for actions in $\rho(s) \subseteq \mathcal{A}(s)$

This can dramatically reduce the branching factor of our search!

3.2 Pruned State Space

Pruned State Space

Plain English:
The pruned state space is what we get when we apply our action-pruning function - it's the search space with some actions removed.

Formal Definition:
Let Π be a planning task, and let $\Theta = (S, A, \text{cost}, T, I, S_G)$ be the state space of Π.
Let ρ be an action-pruning function for Π.
The **pruned state space**, denoted Θ_ρ, is defined as Θ but with T reduced to only those transitions $s \xrightarrow{a} s'$ where $a \in \rho(s)$.

Breaking Down the Notation:

- Θ = Original state space
- T = Set of all transitions in the original space
- $s \xrightarrow{a} s'$ = A transition from state s to state s' via action a
- Θ_ρ = Pruned state space (subset of original)
- $a \in \rho(s)$ = Action a is in the pruned set for state s

Intuition:
Think of the state space as a graph:

- Nodes = states
- Edges = actions

Pruning removes some edges (actions) from the graph while keeping all nodes (states).

Visual Analogy:
Original space: 10 roads from state s
Pruned space: Only 3 roads from state s (the other 7 are "closed")

3.3 Admissible Action-Pruning Functions

Admissibility for Action-Pruning

Plain English:
An action-pruning function is admissible if it doesn't break optimality - we can still find optimal solutions in the pruned space.

Formal Definition:
Let Π be a planning task with state space $\Theta = (S, A, \text{cost}, T, I, S_G)$.
Let ρ be an action-pruning function for Π.
We say that ρ is **admissible** if, for all states $s \in S$:

$$\text{The cost of an optimal solution for } s \text{ in } \Theta_\rho \text{ equals } h^*(s)$$

In Plain English:
Even in the pruned space (with fewer actions), the shortest path from s to the goal has the same cost as in the original space.

What This Means:

- We can throw away some actions
- But we keep at least one optimal path for every state
- So we can still find optimal solutions!

3.4 The Key Theorem for Admissibility

Sufficient Condition for Admissibility

Theorem:
Let Π be a planning task with states S, and let ρ be an action-pruning function for Π. If, for every solvable non-goal state $s \in S$, $\rho(s)$ contains at least one action starting a **shortest optimal plan** for s, then ρ is admissible.

Plain English Translation:
As long as we keep at least one "first step" of some shortest optimal plan at every state, we're guaranteed to find an optimal solution!

Why This Makes Sense:

- If we always have at least one optimal first action available...
- Then we can always take one step toward an optimal solution...
- And after that step, we're in a new state where we again have at least one optimal first action...
- Continuing this way, we'll eventually reach the goal optimally!

Why "Shortest"?
Because we might have 0-cost cycles. If we're in a 0-cost cycle, we need to eventually take an action that breaks out of it. "Shortest" ensures we don't get stuck cycling forever. Example: If we have free teleport actions, we could cycle A→B→A infinitely at zero cost. A shortest optimal plan would avoid such cycles.

Proof Sketch:
By induction on plan length n:
Base case ($n = 1$): If the optimal plan has length 1, we keep that single action by definition, so we can reach the goal in one step.
Inductive step ($n \to n + 1$):

- Let $\pi = \langle a_1, a_2, \ldots, a_n \rangle$ be a shortest optimal plan for s
- By our condition, $\rho(s)$ contains a_1 (the first action) - so we can take this step!
- After applying a_1, we're in state $s' = s[\![a_1]\!]$
- The remaining plan $\langle a_2, \ldots, a_n \rangle$ is a shortest optimal plan for s' (length $n - 1$)
- By induction hypothesis, s' also has an optimal path in Θ_ρ (the pruned space)
- Therefore, s has an optimal path in Θ_ρ: take a_1, then follow the optimal path from s'

What About Unsolvable States?
For unsolvable states, ρ can return anything (even $\emptyset$). It doesn't matter because there's no solution anyway! The theorem only requires the condition for *solvable* states.

The Key Takeaway: This theorem tells us what we need to prove to show that a pruning method is safe. For Strong Stubborn Sets, we'll show they satisfy exactly this property!

4 Strong Stubborn Sets: Ingredients (The Building Blocks)

4.1 The Running Example: 1/2-Log

The 1/2-Log Domain

Setup:
We have a simplified logistics problem:

- 2 trucks (truck 1 and truck 2)
- 2 packages (pack 1 and pack 2)
- 4 locations arranged in a line: A – B – C – D

Variables:

- $\text{truck}_1, \text{truck}_2 \in \{A, B, C, D\}$ - locations of trucks
- $\text{pack}_1, \text{pack}_2 \in \{A, B, C, D, T_1, T_2\}$ - locations of packages (can be at locations or in trucks)

Actions:

- $\text{drive}(i, x, y)$ where $x \neq y$ are neighbors:
  - pre: $\text{truck}_i = x$
  - eff: $\text{truck}_i = y$
- $\text{load}(i, x)$:
  - pre: $\text{pack}_i = x$ and $\text{truck}_i = x$
  - eff: $\text{pack}_i = T_i$
- $\text{unload}(i, x)$:
  - pre: $\text{pack}_i = T_i$ and $\text{truck}_i = x$
  - eff: $\text{pack}_i = x$

Important: Package i can ONLY be loaded/unloaded by truck i!

Typical Problem:
Initial: All packages and trucks at A
Goal: Both packages at D

4.2 Action Dependencies: Enabling

Enabling

Intuition:
Action a_1 **enables** action a_2 if doing a_1 makes a_2 possible (when it wasn't before).

Formal Definition:
Let Π be a planning task with actions $\mathcal{A}$ and states S. Let $a_1 \neq a_2 \in \mathcal{A}$ and $s \in S$.
We say that a_1 **enables** a_2 in s if:

- $a_1 \in \mathcal{A}(s)$ (action 1 is applicable in s)
- $a_2 \notin \mathcal{A}(s)$ (action 2 is NOT applicable in s)
- BUT $a_2 \in \mathcal{A}(s[\![a_1]\!])$ (action 2 IS applicable after doing action 1)

Breaking Down the Notation:

- $s[\![a_1]\!]$ = The state resulting from applying action a_1 in state s (recall from Lecture 4: apply action's effects)
- $\mathcal{A}(s)$ = Set of applicable actions in state s (actions whose preconditions are satisfied)

Plain English:
"Doing a_1 unlocks the ability to do a_2"
Think of it like unlocking a door: you need the key (action a_1) before you can open the door (action a_2).

Why This Matters for POR:
If a_1 enables a_2, then we CANNOT just ignore a_1 and do a_2 directly - we have a necessary ordering! Enabling relationships create dependencies that we must respect when pruning.

Example: Enabling in 1/2-Log

Initial State: All trucks and packages at A

Question: What enables what?

Answer:
drive$(1, A, B)$ enables drive$(1, B, C)$
Why?

- Initially at A: Can do drive(1,A,B) ✓
- Initially at A: CANNOT do drive(1,B,C) × (truck not at B!)
- After drive(1,A,B): Truck at B, so NOW can do drive(1,B,C) ✓

Note: This enabling relationship works in ANY state where drive(1,A,B) is applicable, not just the initial state!

4.3 Action Dependencies: Disabling

Disabling

Intuition:
Action a_1 **disables** action a_2 if doing a_1 makes a_2 impossible (when it was possible before).

Formal Definition:
Let Π be a planning task with actions $\mathcal{A}$ and states S. Let $a_1 \neq a_2 \in \mathcal{A}$ and $s \in S$.
We say that a_1 **disables** a_2 in s if:

- $a_1 \in \mathcal{A}(s)$ (action 1 is applicable in s)
- $a_2 \in \mathcal{A}(s)$ (action 2 is ALSO applicable in s)
- BUT $a_2 \notin \mathcal{A}(s[\![a_1]\!])$ (action 2 is NOT applicable after doing action 1)

Plain English:
"Both actions are currently possible, but doing a_1 destroys the ability to do a_2"

Example: Disabling in 1/2-Log

Initial State: Truck 1 at A, Package 1 at A

Question: What disables what?

Answer:
drive$(1, A, B)$ disables load$(1, A)$
Why?

- At A: Can do drive(1,A,B) ✓
- At A: Can do load(1,A) ✓ (truck and package both at A)
- After drive(1,A,B): CANNOT do load(1,A) × (truck moved to B, package still at A!)

Intuition:
Moving the truck away from the package destroys the possibility of loading the package at A.

4.4 Action Dependencies: Conflict

Conflict

Intuition:
Actions a_1 and a_2 **conflict** if they can both be done in either order, BUT the order matters for the final result.

Formal Definition:
Let Π be a planning task with actions $\mathcal{A}$ and states S. Let $a_1 \neq a_2 \in \mathcal{A}$ and $s \in S$.
We say that a_1 and a_2 **conflict** in s if:

- $a_1 \in \mathcal{A}(s)$ and $a_2 \in \mathcal{A}(s)$ (both applicable in s)
- $a_1 \in \mathcal{A}(s[\![a_2]\!])$ (action 1 still applicable after doing 2)
- $a_2 \in \mathcal{A}(s[\![a_1]\!])$ (action 2 still applicable after doing 1)
- BUT $s[\![\langle a_1, a_2\rangle]\!] \neq s[\![\langle a_2, a_1\rangle]\!]$ (different outcomes!)

Breaking Down the Notation:

- $s[\![\langle a_1, a_2\rangle]\!]$ = State after doing a_1 then a_2
- $s[\![\langle a_2, a_1\rangle]\!]$ = State after doing a_2 then a_1

Plain English:
"Both actions can be done in either order, but the order matters for what we end up with"

Example: Conflict in 1/2-Tele-Log

Modified Domain: Add a teleport action!
teleport(i, y): Instantly move truck i to location y (no preconditions!)

Initial State: Truck 1 at A

Conflicting Actions:
teleport$(1, B)$ and teleport$(1, C)$ conflict!
Why?

- Both applicable at A ✓
- After teleport(1,B): Can still do teleport(1,C) ✓
- After teleport(1,C): Can still do teleport(1,B) ✓
- After ⟨teleport(1,B), teleport(1,C)⟩: Truck at C
- After ⟨teleport(1,C), teleport(1,B)⟩: Truck at B
- C $\neq$ B - Different outcomes!

4.5 Action Dependencies: Interfering

Interfering

Intuition:
Actions **interfere** if they either conflict OR one disables the other.

Formal Definition:
Let Π be a planning task with actions $\mathcal{A}$ and states S. Let $a_1 \neq a_2 \in \mathcal{A}$.
We say that a_1 and a_2 **interfere** if there exists a state $s \in S$ such that:

- a_1 and a_2 conflict in s, OR
- a_1 disables a_2 in s, OR
- a_2 disables a_1 in s

Plain English:
"The actions somehow mess with each other - either by giving different results depending on order, or by one making the other impossible"

Why This Matters:
Interfering actions are the ones we CANNOT ignore! If two actions interfere, we need to keep both of them in our stubborn set (coming later).

Examples of Interfering Actions

From 1/2-Log:

1. $\text{drive}(1, A, B)$ interferes with $\text{load}(1, A)$
   Reason: drive disables load

2. $\text{teleport}(1, B)$ interferes with $\text{teleport}(1, C)$
   Reason: They conflict (different outcomes)

Non-Interfering Actions:
$\text{load}(1, A)$ and $\text{load}(2, A)$ do NOT interfere
Reason: They operate on different packages, neither disables the other, and they don't conflict.

4.6 Action Dependencies: Commutative

Commutative Actions - The Target of POR!

Intuition:
Actions are **commutative** if they can be executed in any order without any issues - they don't interfere and they don't enable each other.

Formal Definition:
Let Π be a planning task with actions $\mathcal{A}$ and states S. Let $a_1 \neq a_2 \in \mathcal{A}$.
We say that a_1 and a_2 are **commutative** if:

- They do NOT interfere (neither conflicts nor disables the other)
- Neither enables the other in any state $s \in S$

Plain English:
"These actions are completely independent - you can do them in any order and it doesn't matter at all"

Concrete Example - Socks and Shoes:

- sockL and sockR are commutative (neither interferes with nor enables the other)
- sockL and shoeL are NOT commutative (sockL enables shoeL - must wear sock before shoe!)
- shoeL and sockR are NOT commutative (they interfere - can't wear shoe and sock simultaneously)

Why This Is The Key to POR:
Commutative actions are the ones we can safely PRUNE! If a_1 and a_2 are commutative and both applicable, we can choose to explore only one of them first, knowing we can always do the other later with the same effect.

The Core Insight of POR: Commutative actions cause plan permutations. By identifying and exploiting commutativity, we can avoid exploring all permutations! This is exactly the problem we saw in the Gripper domain - picking up ball1 then ball2 vs. ball2 then ball1 are commutative actions.

Examples of Commutative Actions

From 1/2-Log:
load$(1, A)$ and load$(2, A)$ are commutative!
Why?

- They don't interfere (operate on different packages)
- Neither enables the other (both depend on their respective trucks being at A)
- Can be done in any order with the same result

More Generally:

load$(1, x)$ and load$(2, y)$ are commutative for any locations x, y
This is the source of the exponential explosion in logistics-like domains!

4.7 Summary of Action Dependencies

The Complete Picture of Action Relationships

Relationship	Plain English Meaning
Enabling	Action a_1 makes action a_2 possible (unlocks it)
Disabling	Action a_1 makes action a_2 impossible (locks it)
Conflict	Actions give different results depending on order
Interfering	Actions either conflict or one disables the other
Commutative	Actions are completely independent (the good case!)

Key Relationships:

- Interfering = Conflicting OR Disabling
- Commutative = NOT Interfering AND NOT Enabling

What We Want:
Maximize commutative actions we can prune!

What We Must Keep:
Interfering actions must be included together!
Enabling chains must be preserved!

4.8 Necessary Enabling Sets

Necessary Enabling Set - The Bottleneck Concept

Intuition:
A necessary enabling set is a collection of actions where AT LEAST ONE must be used to enable something (either an action or the goal).
Think of it as a "bottleneck" or "checkpoint" - no matter which path you take, you must pass through at least one action in this set.

Formal Definition (For Actions):
Let Π be a planning task with actions $\mathcal{A}$, goal G, and states S.
Given $a \in \mathcal{A}$ and $s \in S$ where $a \notin \mathcal{A}(s)$ (action not applicable), a **necessary enabling set** for a in s, denoted $\mathcal{A}_{s \to^* a}$, is a set of actions such that:

$$\text{For every action sequence } \langle a_1, \ldots, a_n \rangle \text{ applicable in } s \text{ where } a_i = a$$
$$\text{we have } \{a_1, \ldots, a_{i-1}\} \cap \mathcal{A}_{s \to^* a} \neq \emptyset$$

Plain English Translation:
If you want to eventually apply action a starting from state s, you MUST use at least one action from the necessary enabling set $\mathcal{A}_{s \to^* a}$ *before* applying a.

Formal Definition (For Goal):
Given $s \in S$ where $G \not\subseteq s$ (goal not satisfied), a **necessary enabling set** for G in s, denoted $\mathcal{A}_{s \to^* G}$, is a set of actions such that:

$$\text{For every action sequence } \langle a_1, \ldots, a_n \rangle \text{ applicable in } s \text{ that achieves } G$$
$$\text{we have } \{a_1, \ldots, a_n\} \cap \mathcal{A}_{s \to^* G} \neq \emptyset$$

Plain English:
To reach the goal from state s, you MUST use at least one action from $\mathcal{A}_{s \to^* G}$ somewhere in your plan.

Key Observation:

- There can be MANY different necessary enabling sets for the same goal/action
- We only need to pick ONE of them for our stubborn set
- Smaller sets are better (lead to less pruning overhead), but harder to find

Examples of Necessary Enabling Sets

From 1/2-Log:
Initial state I: All packages and trucks at A
Goal G: Both packages at D

Example 1: Necessary Enabling Set for an Action
For action drive$(1, B, C)$ in initial state I:

$$\mathcal{A}_{I \to^* \text{drive}(1,B,C)} = \{\text{drive}(1, A, B), \text{teleport}(1, B)\}$$

Why? To be able to drive from B to C, truck 1 must first be at B. The only ways to get there are:

- Drive from A to B
- Teleport to B

You MUST do one of these first!

Example 2: Necessary Enabling Sets for Goal
For goal G (both packages at D) in initial state I:
Set 1: $\mathcal{A}_{I \to^* G} = \{\text{unload}(1, D)\}$
Why? Package 1 must eventually be unloaded at D to satisfy the goal. This is a bottleneck!

Set 2: $\mathcal{A}_{I \to^* G} = \{\text{load}(1, A)\}$
Why? Package 1 must be loaded somewhere to eventually get to D. Loading at A is necessary (if we don't use this, we'd have to load elsewhere, but that requires getting the package elsewhere first).

Set 3: $\mathcal{A}_{I \to^* G} = \{\text{drive}(1, C, D), \text{teleport}(1, D)\}$
Why? To unload package 1 at D, truck 1 must reach D. One of these actions must be used!

Important Note:
There can be MANY different necessary enabling sets for the same goal! We can choose any one of them for our stubborn set computation.

Understanding Necessary Enabling Sets

Think of It Like:
A necessary enabling set is a "bottleneck" or "checkpoint" that you MUST pass through.

Analogy:
Imagine driving from City A to City D:

- There are many possible routes
- But ALL routes must cross the Big Bridge
- The Big Bridge is a "necessary enabling set" - you must use it!

Multiple Necessary Enabling Sets:
Just like there might be multiple bridges (Big Bridge or Tunnel), there can be multiple necessary enabling sets. We just need to pick one!

Why This Matters for Stubborn Sets:
If we include a necessary enabling set for the goal in our stubborn set, we GUARANTEE that:

1. We'll make progress toward the goal
2. We won't get stuck with no way forward
3. We maintain optimality

The Challenge:
Computing necessary enabling sets is HARD (PSPACE-complete)! So in practice, we use approximations (coming in Section 5).

5 Strong Stubborn Sets: Theory

5.1 The Intuition Behind Strong Stubborn Sets

The Big Idea of S3

The Goal:
Given a state s, identify a small subset of applicable actions that's "enough" to explore, while safely ignoring all the others.

Key Idea: We want to focus on ONE "thread" of actions leading to the goal, and ignore all commutative alternatives (different orderings of independent actions).

The Three-Part Strategy:

1. **Make Progress:** Include actions that work toward the goal

   How? Include a necessary enabling set for the goal

   Why? Ensures we're not stuck - we have at least one way forward to the goal

2. **Chain Backwards:** Include actions needed to enable our chosen actions

   How? For every non-applicable action in our set, include a necessary enabling set for it

   Why? Creates a connected path from current state to goal - no "dangling" actions we can't reach

3. **Avoid Permutations:** Include actions that interfere with our chosen applicable actions

   How? For every applicable action in our set, include ALL interfering actions

   Why? Prevents pruning actions that would give different results - we can only prune truly commutative actions

Example: 1/2-Log Small
Initial state: Both packages and both trucks at A
Goal: pack_1 at B, pack_2 at B

Strategy: Focus on package 1 first!

1. **Progress (Condition i):** unload(1, B) - achieves pack_1 at B
2. **Chain (Condition ii):** load(1, A) - needed to enable unload(1, B)
3. **Interfering (Condition iii):** drive(1, A, B), unload(1, A) - interfere with load(1, A)

Result: We can safely ignore all actions related to package 2! They're commutative with our package 1 actions (they don't interfere and don't enable each other).

Why This Works: Any optimal plan must use unload(1,B) to achieve pack_1=B. To do that, it must use load(1,A). So we've captured the essential actions for achieving part of the goal, and excluded the commutative permutations related to package 2.

5.2 The Formal Definition of Strong Stubborn Sets

Strong Stubborn Set (S3)

Recall the Goal: We want to identify a small set of actions that's sufficient to explore while maintaining optimality.

Formal Definition:
Let Π be a planning task with actions $\mathcal{A}$, goal G, and states S. Let $s \in S$ be a non-goal state.
A **strong stubborn set** for s is a set $\mathcal{A}_{S3} \subseteq \mathcal{A}$ of actions such that:

(i) Goal Progress Condition:

$$\mathcal{A}_{S3} \text{ contains a necessary enabling set for } G \text{ in } s$$

Plain English: "Include at least one action that's necessary for reaching the goal"
Purpose: Ensures we make progress - at least one path to goal is preserved

(ii) Backward Chaining Condition:

$$\text{For every } a \in \mathcal{A}_{S3} \setminus \mathcal{A}(s), \quad \mathcal{A}_{S3} \text{ contains a necessary enabling set for } a \text{ in } s$$

Plain English: "For every action in our set that's NOT yet applicable, include at least one action that makes it applicable"
Purpose: Creates a "chain" back to currently applicable actions - ensures all actions in the set are reachable
Notation: $\mathcal{A}_{S3} \setminus \mathcal{A}(s)$ = actions in stubborn set that aren't applicable yet

(iii) Interference Condition:

$$\text{For every } a \in \mathcal{A}_{S3} \cap \mathcal{A}(s), \quad \mathcal{A}_{S3} \text{ contains all } a' \in \mathcal{A} \text{ that interfere with } a$$

Plain English: "For every action in our set that IS applicable, include ALL actions that interfere with it"
Purpose: Prevents pruning actions that would give different outcomes - we can only prune truly commutative actions
Notation: $\mathcal{A}_{S3} \cap \mathcal{A}(s)$ = actions in stubborn set that are applicable now

The Three Conditions Together: (i) picks a goal-relevant action, (ii) chains back to actions we can do now, (iii) includes all alternatives that aren't commutative. This captures ONE path to the goal while excluding permutations!

Complete S3 Example: 1/2-Log Small

State: Initial state I - all packages and trucks at A
Goal: pack_1 at B, pack_2 at B

Building the S3 Step by Step:
Step 1 - Condition (i): Include necessary enabling set for goal
Choose: unload(1, B) - necessary for achieving $\text{pack}_1 = \text{B}$

$$\mathcal{A}_{S3} = \{\text{unload}(1, B)\}$$

Step 2 - Condition (ii): unload(1, B) is not applicable! Need to enable it.
Necessary enabling set: load(1, A)

$$\mathcal{A}_{S3} = \{\text{unload}(1, B), \text{load}(1, A)\}$$

Step 3 - Condition (ii) again: Is load(1, A) applicable?
YES! (truck 1 and package 1 both at A)

Step 4 - Condition (iii): load(1, A) is applicable, so include interfering actions
Interfering actions:

- drive(1, A, B) - disables load(1, A) (moving truck away from package)

$$\mathcal{A}_{S3} = \{\text{unload}(1, B), \text{load}(1, A), \text{drive}(1, A, B)\}$$

Step 5 - Condition (iii): Check newly added applicable actions
drive(1, A, B) is applicable. Check for interfering actions:

- No additional interfering actions outside the current set

Final S3:

$$\mathcal{A}_{S3} = \{\text{unload}(1, B), \text{load}(1, A), \text{drive}(1, A, B)\}$$

Pruned S3 (only applicable):
Let's check applicability in initial state I:

- unload(1, B): Requires $\text{pack}_1 = \text{T}_1$ and $\text{truck}_1 = \text{B} \rightarrow$ NOT applicable
- load(1, A): Requires $\text{pack}_1 = \text{A}$ and $\text{truck}_1 = \text{A} \rightarrow$ APPLICABLE ✓
- drive(1, A, B): Requires $\text{truck}_1 = \text{A} \rightarrow$ APPLICABLE ✓

$$\rho_{S3}(I) = \mathcal{A}(I) \cap \mathcal{A}_{S3} = \{\text{load}(1, A), \text{drive}(1, A, B)\}$$

Result: We've pruned all actions related to package 2 and truck 2!

5.3 S3 Pruning Function Definition

S3 Pruning Function

Definition:
Let Π be a planning task with states S.
An action-pruning function ρ_{S3} for Π is called an **S3 pruning function** if, for every non-goal state $s \in S$, there exists a strong stubborn set $\mathcal{A}_{S3}$ for s such that:

$$\rho_{S3}(s) = \mathcal{A}(s) \cap \mathcal{A}_{S3}$$

Breaking Down the Formula:

- $\mathcal{A}(s)$ = All applicable actions in state s
- $\mathcal{A}_{S3}$ = The strong stubborn set we computed
- $\rho_{S3}(s)$ = The actions we'll actually explore (the intersection)

Plain English:
The S3 pruning function returns only those actions that are:

1. In the strong stubborn set (important for maintaining optimality)
2. AND actually applicable in the current state

Why the Intersection?
The stubborn set might include non-applicable actions (due to condition (ii) - backward chaining). We only want to explore applicable actions, so we take the intersection!

For Goal States:
For goal states, $\rho_{S3}(s)$ can be anything (typically $\emptyset$ since we're done).

5.4 The Main Theorem: S3 is Admissible!

Theorem: S3 Pruning Admissibility

Theorem:
Let Π be a planning task, and let ρ_{S3} be an S3 pruning function.
Then ρ_{S3} is **admissible**.

What This Means:
Using S3 pruning preserves optimality! Even though we're pruning many actions (potentially eliminating exponentially many permutations), we can still find optimal solutions!

Why This Is Important:
This theorem is the theoretical foundation that makes S3 useful in practice. Without admissibility, we couldn't use S3 with A* for optimal planning.

Connection to Earlier Theorem: Recall from slide 13 (Section 3.3) that we showed: if $\rho(s)$ contains at least one action starting a shortest optimal plan for every solvable state s, then ρ is admissible.
Our Task: We need to prove that S3 satisfies exactly this condition - that $\rho_{S3}(s)$ always contains at least one optimal first action.

Proof Strategy Overview:

1. Take ANY plan π (not necessarily optimal)
2. Show π shares an action with our stubborn set (condition i ensures this)
3. Show the first shared action is applicable now (condition ii ensures this)
4. Show this action doesn't interfere with earlier actions (condition iii ensures this)
5. Conclude we can move it to the front, so we have an optimal first action!

5.5 Proof of S3 Admissibility - Part 1

Proof Setup and Key Claims

What We Need to Show:
By the theorem on slide 13, it suffices to show that for every solvable non-goal state s, $\rho_{S3}(s)$ contains at least one action starting a shortest optimal plan for s.

Proof Strategy:
Let s be a solvable non-goal state, and let $\mathcal{A}_{S3}$ be a strong stubborn set for s such that $\rho_{S3}(s) = \mathcal{A}(s) \cap \mathcal{A}_{S3}$.
Let $\pi = \langle a_1, \ldots, a_n \rangle$ be ANY plan for s (not necessarily optimal).
We'll prove three key claims:

Claim (a): π shares an action with $\mathcal{A}_{S3}$
That is: $\{a_1, \ldots, a_n\} \cap \mathcal{A}_{S3} \neq \emptyset$

Claim (b): The first shared action a_k is applicable in s
That is: $a_k \in \mathcal{A}(s)$

Claim (c): a_k doesn't interfere with any earlier actions
That is: a_k doesn't interfere with any of $a_1, \ldots, a_{k-1}$

Once we prove these claims, we can show that a_k can be "moved to the front" of the plan, and since $a_k \in \mathcal{A}(s) \cap \mathcal{A}_{S3} = \rho_{S3}(s)$, we're done!

5.6 Proof of S3 Admissibility - Part 2: Claim (a)

Proof of Claim (a)

Claim (a): π shares an action with $\mathcal{A}_{S3}$

Proof:
By condition (i) of strong stubborn sets, $\mathcal{A}_{S3}$ contains a necessary enabling set for G in s.
Let's call this set $\mathcal{A}_{s\to^*G}$.

By the definition of necessary enabling set for goal:
For every action sequence $\langle a_1, \ldots, a_n\rangle$ applicable in s that achieves G, we have:

$$\{a_1, \ldots, a_n\} \cap \mathcal{A}_{s\to^*G} \neq \emptyset$$

Since $\pi = \langle a_1, \ldots, a_n\rangle$ is a plan for s, it achieves G.
Therefore: $\{a_1, \ldots, a_n\} \cap \mathcal{A}_{s\to^*G} \neq \emptyset$
Since $\mathcal{A}_{s\to^*G} \subseteq \mathcal{A}_{S3}$, we have:

$$\{a_1, \ldots, a_n\} \cap \mathcal{A}_{S3} \neq \emptyset$$

Plain English:
Because we included a necessary enabling set for the goal, and every plan must use at least one action from this set, every plan must share at least one action with our stubborn set! ✓

Claim (a) Example

From 1/2-Log Small:
$\mathcal{A}_{S3} = \{\text{unload}(1, B), \text{load}(1, A), \text{drive}(1, A, B), \text{unload}(1, A)\}$

Consider plan:

$$\pi = \langle \text{load}(2, A), \text{drive}(2, A, B), \text{unload}(2, B), \text{load}(1, A), \text{drive}(1, A, B), \text{unload}(1, B)\rangle$$

Shared actions: load(1, A), drive(1, A, B), unload(1, B) ✓
The plan MUST use some actions from $\mathcal{A}_{S3}$ to achieve the goal!

5.7 Proof of S3 Admissibility - Part 3: Claim (b)

Proof of Claim (b)

Claim (b): The first shared action a_k is applicable in s

Let a_k be the first shared action, i.e., the action with the smallest index in $\{a_1, \ldots, a_n\} \cap \mathcal{A}_{S3}$.

Proof by Contradiction:
Assume $a_k \notin \mathcal{A}(s)$ (not applicable).

Then by condition (ii) of strong stubborn sets, $\mathcal{A}_{S3}$ contains a necessary enabling set for a_k in s.
Let's call this set $\mathcal{A}_{s \to^* a_k}$.

By the definition of necessary enabling set for action:
For the action sequence $\langle a_1, \ldots, a_k \rangle$ where a_k appears at position k, we must have:

$$\{a_1, \ldots, a_{k-1}\} \cap \mathcal{A}_{s \to^* a_k} \neq \emptyset$$

This means some action a_i with $i < k$ is in $\mathcal{A}_{s \to^* a_k} \subseteq \mathcal{A}_{S3}$.

But this contradicts our assumption that a_k is the FIRST shared action!

Therefore, our assumption was wrong, and $a_k \in \mathcal{A}(s)$. ✓

Plain English:
If the first shared action wasn't applicable, we would have included actions to enable it (by condition (ii)). Those enabling actions would appear earlier in the plan, contradicting that this was the first shared action!

Claim (b) Example

From previous example:
First shared action: $a_k = \text{load}(1, A)$ (at position 4)

Is it applicable in initial state? YES! ✓

- truck 1 at A ✓
- package 1 at A ✓

5.8 Proof of S3 Admissibility - Part 4: Claim (c)

Proof of Claim (c)

Claim (c): a_k doesn't interfere with any of $a_1, \ldots, a_{k-1}$

Proof by Contradiction:
Assume a_k interferes with some a_i where $i < k$.

From claim (b), we know that $a_k \in \mathcal{A}(s)$ (applicable in s).

By condition (iii) of strong stubborn sets, since $a_k \in \mathcal{A}_{S3} \cap \mathcal{A}(s)$, we have:

$$\mathcal{A}_{S3} \text{ contains all actions that interfere with } a_k$$

Since a_k interferes with a_i, and interference is symmetric (if a_k interferes with a_i, then a_i interferes with a_k), we have:

$$a_i \in \mathcal{A}_{S3}$$

But $i < k$, which contradicts our assumption that a_k is the FIRST shared action!

Therefore, a_k doesn't interfere with any earlier actions. ✓

Plain English:
If the first shared action interfered with any earlier action, we would have included that earlier action in our stubborn set (by condition (iii)). But that would mean there's a shared action before a_k, which contradicts that a_k is first!

Claim (c) Example

First shared action: load(1, A)
Earlier actions in plan: load(2, A), drive(2, A, B), unload(2, B)

Does load(1, A) interfere with any of these? NO! ✓
They all work on different trucks/packages, so they're commutative.

5.9 Proof of S3 Admissibility - Part 5: The Final Step

Completing the Proof

What We've Proven So Far:
Given any plan $\pi = \langle a_1, \ldots, a_n \rangle$ for state s:

(a) π shares action a_k with $\mathcal{A}_{S3}$

(b) a_k is applicable in s

(c) a_k doesn't interfere with $a_1, \ldots, a_{k-1}$

Key Lemma:
Let s be a state, and let $\pi_k = \langle a_1, \ldots, a_k \rangle$ be an action sequence applicable in s where $a_k \in \mathcal{A}(s)$.
If a_k doesn't interfere with any of $a_1, \ldots, a_{k-1}$, then:

$$\pi'_k := \langle a_k, a_1, \ldots, a_{k-1} \rangle \text{ is applicable in } s$$

and

$$s[\![\pi_k]\!] = s[\![\pi'_k]\!]$$

Plain English: We can move a_k to the front without changing applicability or outcome!

Proof of Lemma:
Since a_k doesn't interfere with $a_1, \ldots, a_{k-1}$:

- It doesn't disable any of them
- It doesn't conflict with any of them

We can prove by induction that a_k remains applicable after each a_i (since none disable it), and we can swap it with each a_i without changing the outcome (since none conflict with it).
Therefore, we can iteratively move a_k to the front.

Completing the Main Proof:
By the lemma, there exists a permutation of π that starts with a_k.
Since $a_k \in \mathcal{A}(s) \cap \mathcal{A}_{S3} = \rho_{S3}(s)$, we've shown that $\rho_{S3}(s)$ contains an action starting a plan for s.
Since this works for ANY plan (including shortest optimal plans), ρ_{S3} is admissible! $\square$

6 Strong Stubborn Sets: Practice (Making It Work)

6.1 The Algorithm: From Definition to Implementation

S3 Computation Algorithm

Input: Planning task Π, state s (non-goal)
Output: Strong stubborn set $\mathcal{A}_{S3}$ for s

```
function ComputeS3(Π, s)
    A_S3 ← A_{s→*G}                          ▷ (i) Necessary enabling set for goal
    Done ← ∅                                 ▷ Actions already processed
    while A_S3 ⊈ Done do
        Select some a ∈ A_S3 \ Done
        if a ∉ A(s) then                     ▷ Action not applicable
            A_S3 ← A_S3 ∪ A_{s→*a}           ▷ (ii) Add necessary enabling set
        else                                 ▷ Action is applicable
            A_S3 ← A_S3 ∪ {a' | a and a' interfere}   ▷ (iii) Add interfering actions
        end if
        Done ← Done ∪ {a}
    end while
    return A_S3
end function
```

How It Works:

1. Start with a necessary enabling set for the goal
2. Repeatedly process actions in the set:
   - If not applicable: Add actions to enable it
   - If applicable: Add interfering actions
3. Continue until all actions processed

Correctness: The algorithm implements exactly the three conditions of the S3 definition!

6.2 The Two Big Challenges for Implementation

Implementation Challenges

To make the algorithm work, we need to solve two problems:

Challenge 1: Finding Interfering Actions
Given an action a, how do we find all actions a' that interfere with a?

Solution: Use syntactic characterization (we can check preconditions and effects)
Status: ✓ Solvable efficiently (coming up!)

Challenge 2: Finding Necessary Enabling Sets
Given state s and action/goal x, how do we find a necessary enabling set $\mathcal{A}_{s \to^* x}$?

Problem: This is PSPACE-complete! (as hard as planning itself)
Solution: Use approximations - find sets that are sufficient (but maybe not necessary)
Status: **!** Must approximate (coming up!)

6.3 Syntactic Characterization of Disabling

Checking for Disabling Syntactically

Terminology:
In an FDR task, partial assignments p and q **agree** if:

$$p(v) = q(v) \text{ for all } v \in \mathcal{V}(p) \cap \mathcal{V}(q)$$

Otherwise, they **disagree**.

Theorem (Syntactic Disabling):
Let Π be an FDR planning task. Let $a_1, a_2 \in \mathcal{A}$.
Then there exists a state s such that a_1 disables a_2 in s if and only if:

1. pre_{a_1} and pre_{a_2} **agree**, AND
2. eff_{a_1} and pre_{a_2} **disagree**

Breaking Down the Conditions:
Condition 1 - Plain English:
"The actions must be applicable together" (no conflicting preconditions)

Condition 2 - Plain English:
"Action 1's effects destroy action 2's preconditions"

Example:
$a_1 = \text{drive}(1, A, B)$:

- pre: $\text{truck}_1 = \text{A}$
- eff: $\text{truck}_1 = \text{B}$

$a_2 = \text{load}(1, A)$:

- pre: $\text{truck}_1 = \text{A}$, $\text{pack}_1 = \text{A}$
- eff: $\text{pack}_1 = \text{T}_1$

Check:

1. pre_{a_1} and pre_{a_2} agree? YES ✓ (both say $\text{truck}_1 = \text{A}$)
2. eff_{a_1} and pre_{a_2} disagree? YES ✓ ($\text{B} \neq \text{A}$)

Therefore, drive(1,A,B) disables load(1,A)!

6.4 Syntactic Characterization of Conflict

Checking for Conflict Syntactically

Theorem (Syntactic Conflict):
Let Π be an FDR planning task. Let $a_1, a_2 \in \mathcal{A}$.
Then there exists a state s such that a_1 and a_2 conflict in s if and only if:

1. pre_{a_1} and pre_{a_2} **agree**, AND
2. eff_{a_1} and pre_{a_2} **agree**, AND
3. eff_{a_2} and pre_{a_1} **agree**, AND
4. eff_{a_1} and eff_{a_2} **disagree**

Breaking Down the Conditions:
Condition 1: "Actions can be applied together initially"
Condition 2: "Action 1's effects don't disable action 2"
Condition 3: "Action 2's effects don't disable action 1"
Condition 4: "Actions have conflicting effects" (different outcomes!)

Example (1/2-Tele-Log):
$a_1 = \text{teleport}(1, B)$:

- pre: (none)
- eff: $\text{truck}_1 = \text{B}$

$a_2 = \text{teleport}(1, C)$:

- pre: (none)
- eff: $\text{truck}_1 = \text{C}$

Check:

1. pre agree? YES ✓ (no preconditions)
2. eff_{a_1} and pre_{a_2} agree? YES ✓ (no preconditions to violate)
3. eff_{a_2} and pre_{a_1} agree? YES ✓ (no preconditions to violate)
4. eff disagree? YES ✓ ($\text{B} \neq \text{C}$)

Therefore, teleport(1,B) and teleport(1,C) conflict!

6.5 Finding Interfering Actions: The Complete Picture

How to Find All Interfering Actions

Given: FDR planning task Π, action a
Goal: Find all actions a' that interfere with a

Recall: Interfering = (Conflicting OR Disabling)

Algorithm:

```
function FindInterfering(Π, a)
    Interfering ← ∅
    for each action a′ ∈ 𝒜 where a′ ≠ a do
        if a and a′ conflict (check 4 conditions) then
            Interfering ← Interfering ∪ {a′}
        else if a disables a′ (check 2 conditions) then
            Interfering ← Interfering ∪ {a′}
        else if a′ disables a (check 2 conditions) then
            Interfering ← Interfering ∪ {a′}
        end if
    end for
    return Interfering
end function
```

Complexity:

- Need to check all actions: $O(|\mathcal{A}|)$
- Each check is polynomial in action size
- **Total: Polynomial!** ✓

Practical Optimization:
Precompute interference graph once for the entire task, then just look up neighbors!

6.6 The Hard Part: Necessary Enabling Sets

The Fundamental Difficulty

Theorem (PSPACE-Completeness):
Let Π be a planning task with actions $\mathcal{A}$, goal G, and states S.

Decision Problem 1:
Given $a \in \mathcal{A}$, $s \in S$ where $a \notin \mathcal{A}(s)$, and $\mathcal{A}' \subseteq \mathcal{A}$, is $\mathcal{A}'$ a necessary enabling set for a in s?
PSPACE-complete!

Decision Problem 2:
Given $s \in S$ where $G \not\subseteq s$ and $\mathcal{A}' \subseteq \mathcal{A}$, is $\mathcal{A}'$ a necessary enabling set for G in s?
PSPACE-complete!

What This Means:
Checking whether a set is necessary enabling is as hard as planning itself! We cannot hope to compute exact necessary enabling sets efficiently.

The Solution: Approximation
Instead of finding necessary enabling sets, we find **sufficient** sets - sets that are guaranteed to enable the action/goal, even if they might be larger than necessary.

Key Insight: Using a sufficient (but not necessary) set is SAFE! It maintains admissibility, just might make the stubborn set larger than needed.

6.7 Approximating Necessary Enabling Sets

Sufficient Enabling Sets - Simple Approach

Theorem (Sufficient Enabling Sets):
Let Π be an FDR planning task. Let $a \in \mathcal{A}$, $s \in S$ where $a \notin \mathcal{A}(s)$.

Let $p \in \text{pre}_a \setminus s$ be a fact that's required by a but not true in s.

Then:

$$\mathcal{A}' = \{a' \in \mathcal{A} \mid p \in \text{eff}_{a'}\}$$

is a necessary enabling set for a in s.

Plain English:
Pick any missing precondition fact p of action a. All actions that achieve p form a necessary enabling set!

Why This Works:
To make a applicable, we MUST achieve all its preconditions. For the chosen fact p, at least one action achieving p must be used.

Same for Goal:
Let $p \in G \setminus s$ be a goal fact not true in s.
Then:

$$\mathcal{A}' = \{a' \in \mathcal{A} \mid p \in \text{eff}_{a'}\}$$

is a necessary enabling set for G in s.

Example: Computing Sufficient Enabling Sets

1/2-Log Example:
Initial state I: $\text{truck}_1 = \text{A}$, $\text{truck}_2 = \text{A}$, $\text{pack}_1 = \text{A}$, $\text{pack}_2 = \text{A}$
Goal G: $\text{pack}_1 = \text{D}$, $\text{pack}_2 = \text{D}$

Example 1: Enabling Set for Action
Action: unload(1, D)
Preconditions:

- $\text{pack}_1 = \text{T}_1$ (NOT true in I) $\times$
- $\text{truck}_1 = \text{D}$ (NOT true in I) $\times$

Choose missing fact: $\text{pack}_1 = \text{T}_1$
Necessary enabling set:

$$\mathcal{A}_{I \to^* \text{unload}(1,D)} = \{\text{load}(1, A), \text{load}(1, B), \text{load}(1, C), \text{load}(1, D)\}$$

All actions that achieve $\text{pack}_1 = \text{T}_1$!

Example 2: Enabling Set for Goal
Goal fact not satisfied: $\text{pack}_1 = \text{D}$
Necessary enabling set:

$$\mathcal{A}_{I \to^* G} = \{\text{unload}(1, D)\}$$

The only action that achieves $\text{pack}_1 = \text{D}$!

Alternative Choice:
Goal fact: $\text{pack}_2 = \text{D}$
Necessary enabling set:

$$\mathcal{A}_{I \to^* G} = \{\text{unload}(2, D)\}$$

Note: We can choose which fact to focus on! This choice affects the stubborn set we get.

6.8 Choosing Which Fact to Focus On

The Open Subgoal Selection Problem

The Issue:
When computing necessary enabling sets, we have a choice:

- For action a with missing preconditions: Which precondition fact to choose?
- For goal G with missing facts: Which goal fact to choose?

Different choices lead to different stubborn sets!

Research Result [Wehrle & Helmert 2014]:
The choice of which fact to focus on can dramatically affect performance!

Two Main Strategies:

1. **Static Strategy:**
   Fix an ordering over FDR state variables once at the beginning.
   Always select the first missing fact according to this ordering.
   Advantage: Consistency across states leads to better caching/learning

2. **Dynamic Strategy:**
   Choice depends on current state s and actions already in $\mathcal{A}_{S3}$.
   Example: Choose the fact that adds the fewest new actions to $\mathcal{A}_{S3}$.
   Advantage: Can lead to smaller stubborn sets

Bottom Line:
Both strategies work (maintain admissibility). The best choice is domain-dependent and requires empirical evaluation.

6.9 Complete S3 Example: Three Little Pigs

The Three Little Pigs Problem - Complete Walkthrough

Domain:
The wolf wants to eat three pigs. He must:

1. Blow down each pig's house
2. Capture each pig
3. Have a banquet once all pigs are captured

Variables:

- $H_1, H_2, H_3 \in \{\text{Fine}, \text{Broken}\}$ - house status
- $P_1, P_2, P_3 \in \{\text{Home}, \text{Captured}\}$ - pig status
- $W \in \{\text{Hungry}, \text{Happy}\}$ - wolf status

Initial State:
$H_1 = H_2 = H_3 = \text{Fine}$, $P_1 = P_2 = P_3 = \text{Home}$, $W = \text{Hungry}$

Goal:
$W = \text{Happy}$

Actions:

- Blow(x): pre $H_x = \text{Fine}$, eff $H_x = \text{Broken}$
- Capture(x): pre $P_x = \text{Home}$, $H_x = \text{Broken}$, eff $P_x = \text{Captured}$
- Banquet: pre $P_1 = P_2 = P_3 = \text{Captured}$, eff $W = \text{Happy}$

Three Little Pigs - Computing S3 for Initial State

Step 1: Condition (i) - Necessary Enabling Set for Goal
Goal: $W = \text{Happy}$
Missing in I: $W = \text{Happy}$

Actions achieving this:

$$\mathcal{A}_{I \to^* G} = \{\text{Banquet}\}$$

$$\mathcal{A}_{S3} = \{\text{Banquet}\}$$

Step 2: Condition (ii) - Enable Banquet
Banquet is not applicable! (pigs not captured)
Missing precondition: Choose $P_1 = \text{Captured}$

Actions achieving this:

$$\mathcal{A}_{I \to^* \text{Banquet}} = \{\text{Capture}(1)\}$$

$$\mathcal{A}_{S3} = \{\text{Banquet}, \text{Capture}(1)\}$$

Step 3: Condition (ii) - Enable Capture(1)
Capture(1) is not applicable! (house not blown)
Missing precondition: $H_1 = \text{Broken}$

Actions achieving this:

$$\mathcal{A}_{I \to^* \text{Capture}(1)} = \{\text{Blow}(1)\}$$

$$\mathcal{A}_{S3} = \{\text{Banquet}, \text{Capture}(1), \text{Blow}(1)\}$$

Step 4: Check Blow(1)
Blow(1) IS applicable in I! ✓
No more enabling needed for this action.

Three Little Pigs - Computing S3 (continued)

Step 5: Condition (iii) - Interfering Actions for Blow(1)
Blow(1) is applicable. Find all actions that interfere with it.

Check all actions:

- Blow(2), Blow(3): Operate on different houses - NO interference ✓
- Capture(1): Requires $H_1 = \text{Broken}$, Blow(1) achieves this - NO conflict ✓
- Capture(2), Capture(3): Operate on different pigs - NO interference ✓
- Banquet: No shared preconditions/effects - NO interference ✓

No interfering actions! (Blowing houses are independent)

$$\mathcal{A}_{S3} = \{\text{Banquet}, \text{Capture}(1), \text{Blow}(1)\}$$

Step 6: Check Capture(1) and Banquet
Both are not applicable, already processed for enabling.

Final Stubborn Set:

$$\mathcal{A}_{S3} = \{\text{Blow}(1), \text{Capture}(1), \text{Banquet}\}$$

Pruned Set (only applicable):

$$\rho_{S3}(I) = \mathcal{A}(I) \cap \mathcal{A}_{S3} = \{\text{Blow}(1)\}$$

What Got Pruned:
All actions for pigs 2 and 3! (Blow(2), Blow(3), Capture(2), Capture(3))

Why This Works:
We focused on pig 1 first. Actions for pigs 2 and 3 are commutative with pig 1 actions, so we can safely ignore them for now!

6.10 How Much Does the Choice Matter?

Three Little Pigs - Different Choices

Choice 1: Focus on P_1 = Captured (as above)
Result: $\rho_{S3}(I) = \{\text{Blow}(1)\}$ (1 applicable action)

Choice 2: Focus on P_1 = Captured AND P_2 = Captured
Step 1: $\mathcal{A}_{S3} = \{\text{Banquet}\}$
Step 2a: Add Capture(1) for P_1 = Captured
Step 2b: Add Capture(2) for P_2 = Captured
Step 3a: Add Blow(1) to enable Capture(1)
Step 3b: Add Blow(2) to enable Capture(2)

Result: $\rho_{S3}(I) = \{\text{Blow}(1), \text{Blow}(2)\}$ (2 applicable actions)

Choice 3: Include all three pigs' preconditions
Result: $\rho_{S3}(I) = \{\text{Blow}(1), \text{Blow}(2), \text{Blow}(3)\}$ (3 applicable actions)

Observation:

- All three choices are valid S3s (admissible!) ✓
- Smaller sets = more pruning = potentially faster search
- But different problems benefit from different strategies

Practical Advice: Use static strategy for consistency, or dynamic strategy to minimize stubborn set size. Both work!

7 Summary and Key Takeaways

7.1 The Big Picture: What We Learned

Course Red Thread Summary

The Journey So Far:

1. **Lectures 2-3:** Uninformed and heuristic search
   - A* finds optimal solutions using admissible heuristics
   - Performance depends on heuristic quality
2. **Lectures 4-5:** Better heuristics through abstraction
   - Delete relaxation (Lecture 4) and Pattern Databases (Lecture 5)
   - Can achieve almost-perfect heuristics!
3. **Lecture 6 (This Lecture):** Even perfect heuristics aren't enough!
   - Problem: Plan permutations cause exponential blow-up
   - Solution: Partial Order Reduction (POR)

The Problem We Discovered:
Even with almost-perfect heuristics (error ≤ 1), we can have exponential search space due to **plan permutations**.

Why Permutations Happen:
Commutative actions - actions that can be executed in any order without affecting the outcome (e.g., picking up ball1 vs ball2 in Gripper).

The Solution: Partial Order Reduction (POR)
A family of **action pruning** techniques that identify and prune permutable parts of the search space.

This Lecture's Focus: Strong Stubborn Sets (S3)
A state-reduction POR method that:

- Identifies a small subset of actions to explore at each state
- Maintains **optimality** (admissible!)
- Works *orthogonally* to heuristics - they complement each other!
  - Heuristics: Guide search toward goal
  - S3: Prune different orderings of actions

The Big Picture: Modern optimal planning uses BOTH techniques: powerful heuristics (Lectures 4-5) to guide search toward goals, and POR (Lecture 6) to avoid exploring equivalent permutations. Together, they tackle search from two orthogonal dimensions!

7.2 Key Concepts Recap

Action Dependencies

Five key relationships between actions:

1. **Enabling:** a_1 makes a_2 possible
   Example: drive(1,A,B) enables drive(1,B,C)

2. **Disabling:** a_1 makes a_2 impossible
   Example: drive(1,A,B) disables load(1,A)

3. **Conflict:** Order matters for outcome
   Example: teleport(1,B) vs teleport(1,C)

4. **Interfering:** Conflict OR disabling
   Purpose: Actions we MUST keep together

5. **Commutative:** Completely independent
   Purpose: Actions we CAN prune

The Goal: Maximize pruning of commutative actions while preserving interfering/enabling relationships.

Strong Stubborn Sets Definition

A strong stubborn set $\mathcal{A}_{S3}$ for non-goal state s must satisfy three conditions:

(i) Goal Progress:
Include a necessary enabling set for the goal
Plain English: "Pick actions that work toward the goal"

(ii) Backward Chaining:
For non-applicable actions in $\mathcal{A}_{S3}$, include necessary enabling sets
Plain English: "Chain back to actions we can actually do now"

(iii) Interference:
For applicable actions in $\mathcal{A}_{S3}$, include ALL interfering actions
Plain English: "Don't ignore actions that would give different results"

Key Theorem:
S3 pruning is **admissible** - maintains optimality!

Why It Works:
For any plan π:

- π shares at least one action with $\mathcal{A}_{S3}$ (condition i)
- The first shared action is applicable (condition ii)
- This action doesn't interfere with earlier actions (condition iii)
- Therefore, we can permute it to the front!

Implementation Challenges & Solutions

Challenge 1: Finding Interfering Actions
Solution: Syntactic characterization using preconditions and effects

For disabling: Check if pre agree and eff/pre disagree
For conflict: Check if pre agree, eff/pre agree both ways, but eff disagree

Status: ✓ Polynomial time!

Challenge 2: Finding Necessary Enabling Sets
Problem: PSPACE-complete (as hard as planning!)

Solution: Use *sufficient* enabling sets instead

Simple approach: Pick missing precondition/goal fact p, include all actions achieving p

Trade-off: Approximation may give larger stubborn sets, but maintains correctness!

The Selection Problem:
Which missing fact to choose?

- **Static:** Fixed ordering, consistent across states
- **Dynamic:** Minimize new actions added

Both maintain admissibility; best choice is domain-dependent.

7.3 Practical Guidelines

When to Use S3

S3 Works Best When:

- Problem has many **commutative actions**
- Examples: Logistics, Gripper, Resource allocation, Multi-robot coordination
- Characteristic: Independent subproblems that can be solved in any order
- Signs: Many actions that don't affect each other's preconditions or effects

S3 Provides Less Benefit When:

- Few commutative actions (highly sequential problems)
- Most actions interfere with each other
- Example: Tower of Hanoi (strict ordering required), assembly tasks with dependencies
- In these cases, S3 overhead may outweigh benefits

Combining with Other Techniques (The Power of Integration):

- **S3 + Heuristics (e.g., A* with PDBs):** Works great! Orthogonal approaches
  - Heuristics guide which states to expand
  - S3 reduces actions explored per state
  - Together: Dramatic speedup on logistics-like domains
- **S3 + Symmetry Reduction:** Can be combined
  - Symmetry: Eliminates duplicate states
  - S3: Eliminates action permutations
  - Both attack the search space from different angles
- **S3 + Pattern Databases:** Complementary!
  - PDBs provide strong heuristics
  - S3 handles remaining permutations
  - Best of both worlds for optimal planning

Implementation Tips:

1. Precompute interference graph once (analyze all action pairs offline)
2. Cache stubborn sets when possible (state-to-set mapping)
3. Use static subgoal selection for better caching (consistent choices across states)
4. Monitor stubborn set sizes - if too large (¿50% of applicable actions), may not help
5. Start simple: Implement basic S3, then optimize based on profiling

7.4 Common Pitfalls and How to Avoid Them

Common Mistakes

Mistake 1: Confusing Necessary and Sufficient
Wrong: "This set enables the action, so it must be necessary"
Right: "This set enables the action, so it's sufficient. It might not be necessary (could be smaller), but that's OK!"

Mistake 2: Forgetting to Include Interfering Actions
Wrong: Only including enabling chains, ignoring condition (iii)
Right: MUST include all interfering actions for applicable actions in $\mathcal{A}_{S3}$
Consequence if wrong: Loss of admissibility!

Mistake 3: Thinking Commutativity Means No Shared Variables
Wrong: "Actions on different variables are always commutative"
Right: Must check: no interference AND no enabling
Example: drive(1,A,B) and load(2,A) operate on different trucks/packages but share location A

Mistake 4: Pruning Too Aggressively
Wrong: "Let's just keep one action and prune everything else!"
Right: Must follow the three S3 conditions carefully
Why: Arbitrary pruning can lose optimal solutions!

7.5 Quick Reference: The S3 Algorithm

S3 Computation at a Glance

Input: Planning task Π, non-goal state s
Output: Pruned action set $\rho_{S3}(s)$

Step-by-Step:

1. **Initialize:**
   - Pick missing goal fact $p \in G \setminus s$
   - $\mathcal{A}_{S3} \leftarrow \{a \in \mathcal{A} \mid p \in \text{eff}_a\}$

2. **Process each action in $\mathcal{A}_{S3}$:**
   - If $a \notin \mathcal{A}(s)$: Add sufficient enabling set
     - Pick missing precondition $p \in \text{pre}_a \setminus s$
     - Add $\{a' \in \mathcal{A} \mid p \in \text{eff}_{a'}\}$
   - If $a \in \mathcal{A}(s)$: Add interfering actions
     - Add all a' that conflict with a
     - Add all a' that disable or are disabled by a

3. **Repeat step 2** until no new actions added

4. **Return:** $\rho_{S3}(s) = \mathcal{A}(s) \cap \mathcal{A}_{S3}$

Complexity:
Depends on stubborn set size (typically much smaller than $|\mathcal{A}|$)

7.6 Further Reading and Advanced Topics

- **Wehrle & Helmert (2012):** *About Partial Order Reduction in Planning and Computer Aided Verification*

  Original paper introducing stubborn sets to planning

- **Wehrle & Helmert (2014):** *Efficient Stubborn Sets: Generalized Algorithms and Selection Strategies*

  More general S3 definition and empirical comparison of selection strategies

- **Advanced Topics Not Covered:**
  - Sleep sets (another POR technique)
  - Ample sets (from model checking)
  - Combining S3 with symmetry reduction
  - Weak stubborn sets (for satisficing planning)
  - Expansion core (alternative to S3)

End of Partial Order Reduction Guide

Master the art of pruning plan permutations for optimal search!